

Mastering Policy Gradients & Variance Reduction

Complete Beginner-Friendly Breakdown: The Log-Derivative Trick, EGLP Lemma, Achiam's 5 Forms of π_{θ} , Gaussian Continuous Control & Bi-Manual Slicing

Instructor: Prof. Sergey Levine (UC Berkeley)

Companion: DEX-ROB Lab, Tianjin University

Frameworks: Achiam (Spinning Up) + Goodfellow (Optimization)

0. The "Mental Map": Why Does Lecture 5 Exist?

In Lecture 4, we formulated the general goal of reinforcement learning: find a policy $\pi_{\theta}(a|s)$ that maximizes expected cumulative return $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [r(\tau)]$.

The Central Dilemma of Robotics: In standard deep learning, you compute the gradient of your loss with respect to network weights via backpropagation ($\nabla_{\theta} \mathcal{L}$). But in robotics, between the robot's motors and the final reward lies **the physical universe**: non-linear friction, knife-tissue deformation, skin crack initiation, and fluid dynamics.

You cannot backpropagate a gradient through a physical tomato! How can we optimize a neural network when the physical laws connecting our actions to the results are completely non-differentiable? **Lecture 5 introduces the Policy Gradient Theorem and the magical "Log-Derivative Trick" that solves this dilemma.**

1. Log-Derivative

No Physics Model Needed!

2. Gaussian Policy

Continuous Actions: Mean μ and Noise σ

3. The Variance Trap

Lucky vs Unlucky Noisy Gradients

4. Causality

Reward-to-Go: Past $\neq$ Future

5. Baselines & EGLP

Subtracting $b(s)$: Zero Bias, Low Var

Part 1: REINFORCE & The Log-Derivative Trick

(Slides 1–15, Spoken Transcript 02:40–16:50) How do we compute $\nabla_{\theta} J(\theta)$ without differentiating through the environment transition dynamics $p(s_{t+1}|s_t, a_t)$?

1.1 The Mathematical Derivation (Step-by-Step for Beginners)

A trajectory $\tau = (s_1, a_1, s_2, a_2, \dots, s_T)$ has probability:

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^{T-1} p_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

The expected return is: $J(\theta) = \int p_{\theta}(\tau) r(\tau) d\tau$. Taking the gradient directly: $\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) r(\tau) d\tau$. Using the Log-Derivative Identity $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$:

$$\nabla_{\theta} J(\theta) = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) r(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\nabla_{\theta} \log p_{\theta}(\tau) r(\tau) \right]$$

1.2 Why the Physics Vanishes!

Now expand $\log p_{\theta}(\tau)$: $\log p_{\theta}(\tau) = \log p(s_1) + \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) + \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t)$. Notice that $p(s_1)$ and the transition dynamics $p(s_{t+1} | s_t, a_t)$ do NOT depend on policy weights θ ! Their derivatives with respect to θ are identically zero!

The Policy Gradient Theorem (REINFORCE):

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \left(\sum_{t=1}^T r(s_{i,t}, a_{i,t}) \right)$$

PLAIN-ENGLISH INTUITION: WHAT DID WE JUST DO?

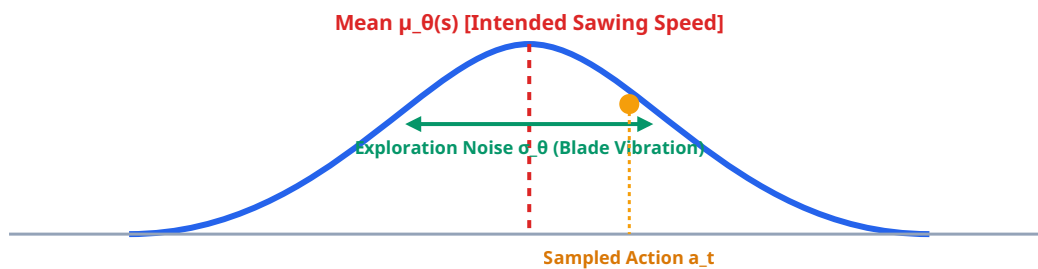
We do NOT need an equation for the skin toughness of a tomato. We only need to run simulated cuts, record which trajectories scored high rewards, and increase the likelihood of the actions the network took during those successful runs. Good actions are pushed to happen more often; bad actions are pushed to happen less often. This is mathematically formalized trial-and-error!

Part 2: Continuous Actions & Gaussian Policies

(Slides 16–22, Spoken Transcript 17:00–28:30) In robotic manipulation, actions are continuous variables: feed velocity $\Delta z \in [-2, 2]$ mm, sawing speed $v_{\text{saw}} \in [-50, 50]$ mm/s, stiffness $\Delta K_z \in [-200, 200]$ N/m.

2.1 The Gaussian Policy Network

$$\pi_{\theta}(a_t | s_t) = \mathcal{N}(\mu_{\theta}(s_t), \Sigma_{\theta}(s_t)) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(a_t - \mu_{\theta}(s_t))^2}{2\sigma^2}\right)$$



2.2 The Analytical Derivative of a Gaussian Policy

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \frac{a - \mu_{\theta}(s)}{\sigma^2}, \quad \nabla_{\theta} \mu_{\theta}(s)$$

PHYSICAL ROBOTICS MEANING: PUSHING THE MEAN

Look closely at the term $(a - \mu_{\theta}(s))$:

- Suppose the network intended to saw at $\mu = 15$ mm/s, but exploratory noise sampled $a = 22$ mm/s ($a - \mu = +7$).
- If this cut successfully sliced the tomato skin without crushing, return $r(\tau)$ is positive ($+10$).
- The gradient update pushes $\mu_{\theta}(s)$ in the direction of $+7$, shifting the mean sawing speed higher!

SPINNING UP BUG ALERT: CLAMPING LOG-STD

Joshua Achiam warns: if you let `log_std` float unconstrained, gradients will cause $\sigma \rightarrow 0$ (destroying exploration prematurely) or $\sigma \rightarrow \infty$ (causing `NaN` in division by σ^2). In production robotics code, always clamp log standard deviations: `log_std = torch.clamp(log_std, min=-20, max=2)`.

Part 3: The High Variance Nightmare

(Slides 23–28, Spoken Transcript 29:00–41:10) While REINFORCE is mathematically elegant, Sergey Levine points out why raw REINFORCE is completely unusable in practical robotics without variance reduction.

3.1 The Chess Analogy (Levine's Example)

In chess (\$+1\$ for win, \$-1\$ for loss): If the agent plays an extraordinary opening at move 5, but blunders at move 45 and loses, REINFORCE multiplies *every* move by \$-1\$, so the brilliant move 5 is penalized! Mathematically, these errors average out over infinite samples, but in practice, you might need **100 million trajectories** just to drown out the noise!

Trajectory 1: Unlucky Contact Slip

Steps 1-170: Perfect adaptive sawing

Step 175: Gripper slip → Total Collapse

Raw REINFORCE: Penalizes ALL 175 steps!

Trajectory 2: Lucky Puncture

Steps 1-50: Terrible erratic stiffness

Step 52: Random puncture → Fruit holds

Raw REINFORCE: Rewards ALL erratic steps!

Part 4: Variance Reduction: Causality & Reward-to-Go

(Slides 29–35, Spoken Transcript 41:30–52:00) Physical causality states: **actions taken in the present cannot affect rewards received in the past!**

4.1 The Causality Proof

For any time step $t' < t$, $\mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot r(s_{t'}, a_{t'})] = 0$.
Discarding past rewards yields the **Reward-to-Go** $\hat{Q}_{i,t}$:

Reward-to-Go Policy Gradient:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right)$$

Part 5: Baselines, EGLP Lemma & The 5 Forms of Φ_t

(Slides 36–45 & Spinning Up ch09) In **Spinning Up in Deep RL**, Joshua Achiam unifies all policy gradient methods into a single universal template:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t), \Phi_t \right]$$

Achiam identifies the **5 valid choices of Φ_t** , proving they all share the exact same mathematical expectation and differ only in variance:

Choice of Φ_t	Mathematical Expression	Variance & Bias Tradeoff
1. Total Return	$\Phi_t = R(\tau) = \sum_{t'=0}^T r_{t'}$	Unbiased, Highest Variance (pure REINFORCE).
2. Reward-to-Go	$\Phi_t = \sum_{t'=t}^T r_{t'}$	Unbiased, Lower Variance (removes past noise via causality).
3. Return with Baseline	$\Phi_t = \sum_{t'=t}^T r_{t'} - b(s_t)$	Unbiased, Significantly Lower Variance (centers returns).
4. State-Action Q	$\Phi_t = Q^{\pi}(s_t, a_t)$	Biased (if Q learned), Very Low Variance .
5. Advantage Function	$\Phi_t = A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$	Biased (if approximated), Lowest Variance (Actor-Critic & PPO).

5.1 The EGLP Lemma (Why Baselines are Unbiased)

$$\text{The Expected Grad-Log-Prob (EGLP) Lemma: } \mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)] = 0$$

Because $\mathbb{E}_{a \sim \pi} [\nabla \log \pi(a|s)] = 0$, any state baseline $b(s)$ factors out: $\mathbb{E} [\nabla \log \pi(a|s) b(s)] = b(s) \cdot 0 = 0$.

ACHIAM'S RULE: THE POLICY GRADIENT LOSS IS NOT A LOSS FUNCTION!

In supervised learning, loss measures performance. In policy gradients, the objective is a surrogate whose distribution shifts with θ . **You can drive the policy gradient surrogate loss to negative infinity while the robot's real cutting performance craters!** Never use policy gradient loss to evaluate convergence. Only average episode return ($\bar{R}$) and success rate are meaningful metrics.

Part 6: Paper 1 PyTorch Implementation Blueprint

```
import torch
import torch.nn as nn
from torch.distributions.normal import Normal

class TomatoCuttingPolicy(nn.Module):
    def __init__(self, state_dim=33, action_dim=6):
        super().__init__()
        self.backbone = nn.Sequential(
            nn.Linear(state_dim, 256), nn.ELU(),
            nn.Linear(256, 128), nn.ELU()
        )
        self.mean_layer = nn.Linear(128, action_dim)  # Learnable
        self.log_std = nn.Parameter(torch.zeros(action_dim))

    def forward(self, state):
        features = self.backbone(state)
        mean = self.mean_layer(features)  # Numerical stability clamp: prevents NaN or zero-variance
        log_std = torch.clamp(self.log_std, min=-20.0, max=2.0)
        std = torch.exp(log_std)
        return Normal(mean, std)

    def compute_policy_gradient_loss(self, policy, states, actions, rewards_to_go, baselines):
        dist = policy(states)
        log_probs = dist.log_prob(actions).sum(dim=-1, keepdim=True)  # CRITICAL: detach baselines so value gradients do not corrupt policy
        advantages = rewards_to_go - baselines.detach()  # Advantage normalization across batch
        advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)  # Negative surrogate loss for PyTorch optimizer minimization
        loss = -(log_probs * advantages).mean()
        return loss
```

Part 7: Interactive Tablet Self-Test Quiz (Test Your Understanding)

Question 1: Why does the environmental physics model $p(s_{t+1}|s_t, a_t)$ vanish when we apply the log-derivative trick?

Answer: When expanding $\log p_{\theta}(\tau)$, the physics dynamics appear as an additive term $\sum \log p(s_{t+1}|s_t, a_t)$. Because the laws of physics do not depend on the neural network parameters θ , taking the partial derivative ∇_{θ} causes this term to differentiate to exactly zero ($\nabla_{\theta} \log p(s_{t+1}|s_t, a_t) = 0$).

Question 2: According to the EGLP Lemma, why does subtracting a state baseline $b(s)$ not change the expectation of the policy gradient?

Answer: The Expected Grad-Log-Prob Lemma states that $\mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a|s)] = 0$ because probability densities integrate to 1. Since $b(s)$ is independent of action a , it factors out as $b(s) \cdot 0 = 0$.

Question 3: If you forget to call `.detach()` on the baseline $V(s)$ when computing advantages, what silent bug occurs?

Answer: The PyTorch autograd graph will backpropagate the policy loss backward through the baseline network. Instead of fitting $V(s)$ to predict returns via MSE, the value network will be distorted by policy gradients, destabilizing value estimation and destroying training.

Part 8: Thesis Defense Master Cheatsheet (Lecture 5 Focus)

Q1: "WHY CAN'T YOU SIMPLY BACKPROPAGATE GRADIENTS DIRECTLY THROUGH THE ISAAC LAB PHYSICS ENGINE?"

Answer: "While differentiable simulators exist, soft contact mechanics—specifically fracture of the tomato skin cuticle, non-linear fluid outflow, and frictional stick-slip transitions—are fundamentally discontinuous and non-differentiable. Even if approximate gradients could be computed, they suffer from catastrophic gradient explosion across multi-step contact impacts. The Policy Gradient Theorem bypasses physical derivatives entirely by differentiating only the policy's log-action probabilities weighted by the scalar performance return."

Q2: "WHAT IS THE PHYSICAL ROLE OF THE POLICY'S STANDARD DEVIATION σ IN ROBOTIC CUTTING?"

Answer: "In our continuous Gaussian policy, σ represents active motor exploration. Early in training, high σ creates stochastic micro-vibrations and multi-axis sawing motions, enabling the robot to stumble upon skin puncture events. As training converges, σ decays naturally, yielding a smooth, deterministic compliant trajectory that executes steady sawing without erratic force oscillations."

Q3: "WHY IS REWARD-TO-GO MATHEMATICALLY SUPERIOR TO TOTAL TRAJECTORY RETURN?"

Answer: "Total return multiplies the action at step t by past rewards received prior to step t . By causality, the robot's current action cannot influence events that have already occurred ($\mathbb{E}[\nabla \log \pi_t \cdot r_{t'}] = 0$ for $t' < t$). Retaining past rewards adds pure noise to the gradient estimate. Reward-to-go discards past rewards, preserving the identical mathematical expectation while dramatically reducing gradient variance."